

main

December 7, 2025

```
[212]: import numpy as np
import pandas as pd
```

Data inladen:

```
[213]: data = pd.read_csv("videos_latest_snapshots_history.csv")
print("Rijen:", len(data))
print("Kolommen:", data.columns.tolist())
data.head(3)
```

Rijen: 732

Kolommen: ['video_id', 'title', 'channel_id', 'channel_title', 'published_at', 'description', 'tags', 'duration', 'dimension', 'definition', 'caption_available', 'licensed_content', 'snapshot_date', 'fetches_at', 'view_count', 'like_count', 'comment_count', 'favorite_count']

```
[213]:      video_id      title \
0  nKd8Rou95dE  Ethics in AI: Biases & Responsibilities • Mich...
1  Zd3KVhSkfb8  Speak at Devovx Belgium 2025: Share Your Ideas...
2  5PaJsEbBARo  Hands-on Demo of Glamorous Toolkit & Moldable ...

      channel_id      channel_title      published_at \
0  UCs_tLP3AiwYKwdUHpltJPuA  GOTO Conferences  2025-09-09 12:57:31
1  UCCBVCTuk6uJrN3iFV_3vurg      Devovx  2025-07-01 10:03:11
2  UCs_tLP3AiwYKwdUHpltJPuA  GOTO Conferences  2025-09-17 12:00:11

      description \
0  This interview was recorded for GOTO Unscripte...
1  The Call for Papers (CFP) for Devovx Belgium 2...
2  This interview was recorded at GOTO Copenhagen...

      tags      duration      dimension \
0  ['GOTO', 'GOTOcon', 'GOTO Conference', 'GOTO (...  PT40M7S      2d
1  []      PT33S      2d
2  ['GOTO', 'GOTOcon', 'GOTO Conference', 'GOTO (...  PT42M55S      2d

      definition      caption_available      licensed_content      snapshot_date \
0      hd      False      True      2025-09-22
1      hd      False      False      2025-09-22
```

	2	hd	False	True	2025-09-22
		fetches_at	view_count	like_count	comment_count \
0	2025-09-22	04:00:03.933320	281	0	0
1	2025-09-22	04:00:03.933251	740	7	0
2	2025-09-22	04:00:03.933091	453	7	1
		favorite_count			
0		0			
1		0			
2		0			

data uitkiezen + engagement score berekenen

```
[214]: views_col    = "view_count"
likes_col    = "like_count"
comments_col = "comment_count"

for c in [views_col, likes_col, comments_col]:
    data[c] = pd.to_numeric(data[c], errors="coerce").fillna(0)

data["engagement_score"] = np.where(
    data[views_col] > 0,
    (data[likes_col] + data[comments_col]) / np.sqrt(data[views_col]) * 100,
    0.0
)
```

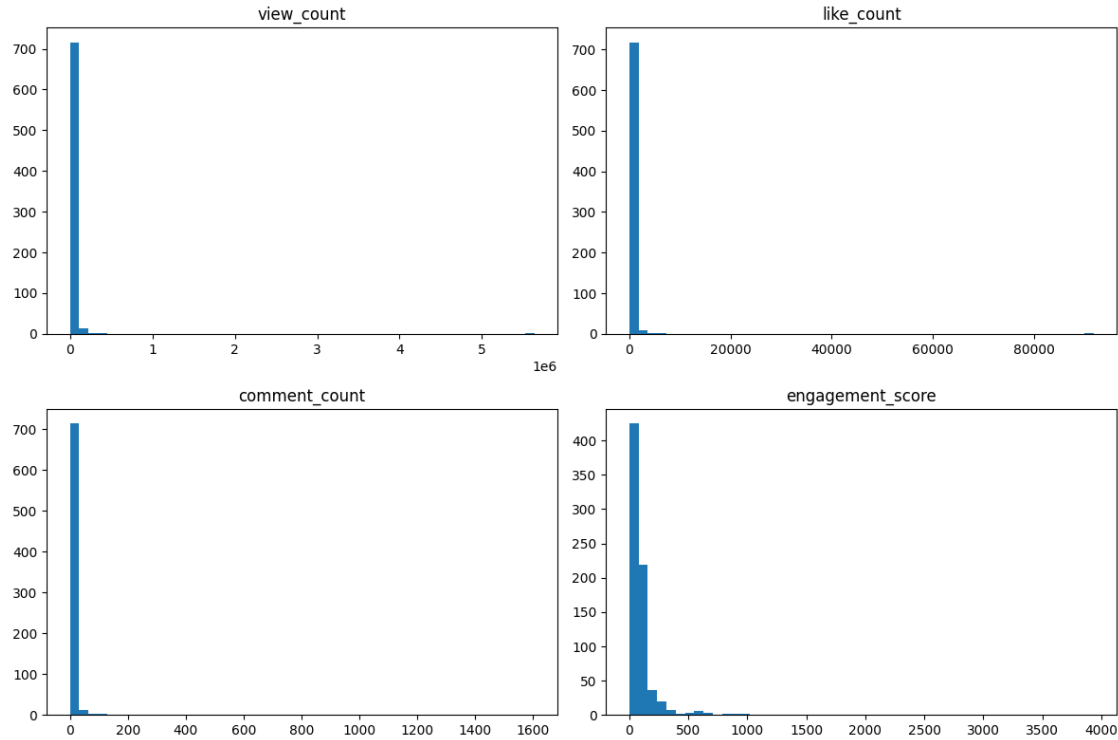
```
[215]: data[["view_count", "like_count", "comment_count", "engagement_score"]].
        describe()

import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
cols = ["view_count", "like_count", "comment_count", "engagement_score"]

for ax, col in zip(axes.flatten(), cols):
    ax.hist(data[col], bins=50)
    ax.set_title(col)

plt.tight_layout()
plt.show()
```



features uitkiezen

```
[216]: features = [
        'engagement_score',
        'view_count',
        'like_count',
        'comment_count'
    ]
```

De ruwe aantallen views, likes en comments zijn sterk scheef verdeeld.

Daarom pas ik:

een log-transformatie (`np.log1p`) toe om extreme waarden in te perken;

winsorizing (1e-96e percentiel) om extreme uitschieters te clippen;

een `RobustScaler` toe om te schalen op basis van median en IQR, zodat het model minder gevoelig is voor outliers.

```
[217]: import numpy as np
        from sklearn.preprocessing import RobustScaler

        X = data[features].astype(float).copy()
```

```

# Missing values
X = X.fillna(0)
X = X.replace([np.inf, -np.inf], 0)

for col in X.columns:
    X[col] = np.log1p(X[col])

# Clip extreme outliers (winsorize 1%-99%)
for col in X.columns:
    lower = X[col].quantile(0.01)
    upper = X[col].quantile(0.96)
    X[col] = X[col].clip(lower, upper)

# Scale after clipping
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)

print("Vorbewerking klaar.")

```

Vorbewerking klaar.

2e trainingsfase, deze keer met gewichten

```

[218]: weights_phase2 = np.array([
        2.0, # engagement_score
        1.5, # view_count
        0.2, # like_count
        0.2  # comment_count
    ])

X_weighted = X_scaled * weights_phase2

```

Kmeans uitvoeren, 3 clusters want er zijn er 3. voor beide fases.

```

[219]: from sklearn.cluster import KMeans

# KMeans clustering
kmeans_phase1 = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=50,
    max_iter=1000
)

# pas KMeans toe en sla labels op
labels_phase1 = kmeans_phase1.fit_predict(X_scaled)
data['cluster_label_phase1'] = labels_phase1

print("Trainingsfase 1 - non-weighted clustering voltooid.")

```

```

kmeans_phase2 = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=50,
    max_iter=1000
)

labels_phase2 = kmeans_phase2.fit_predict(X_weighted)
data["cluster_label_phase2"] = labels_phase2

print("Trainingsfase 2 - weighted clustering voltooid.")

```

Trainingsfase 1 - non-weighted clustering voltooid.

Trainingsfase 2 - weighted clustering voltooid.

labels toewijzen:

```

[220]: X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

tmp = pd.DataFrame({
    "label": labels_phase1,
    "engagement_score": X_scaled_df["engagement_score"]
})

medians_phase1 = tmp.groupby("label")["engagement_score"].median().sort_values()

sorted_clusters = medians_phase1.index.tolist()

label_mapping_phase1 = {
    sorted_clusters[0]: 'Low engagement',
    sorted_clusters[1]: 'Medium engagement',
    sorted_clusters[2]: 'High engagement'
}

data['engagement_cluster'] = data['cluster_label_phase1'].
    ↪map(label_mapping_phase1)

cat_order = ['High engagement', 'Medium engagement', 'Low engagement']
data['engagement_cluster'] = pd.Categorical(
    data['engagement_cluster'], categories=cat_order, ordered=True
)

```

```

[221]: # Label mapping op basis van median engagement_score
tmp2 = pd.DataFrame({
    "label": labels_phase2,
    "engagement_score": data["engagement_score"]
})

```

```

medians_phase2 = tmp2.groupby("label")["engagement_score"].median().
    ↪sort_values()

sorted_clusters2 = medians_phase2.index.tolist()

label_mapping_phase2 = {
    sorted_clusters2[0]: "Low engagement",
    sorted_clusters2[1]: "Medium engagement",
    sorted_clusters2[2]: "High engagement"
}

data["engagement_cluster_phase2"] = (
    data["cluster_label_phase2"]
    .map(label_mapping_phase2)
)

data["engagement_cluster_phase2"] = pd.Categorical(
    data["engagement_cluster_phase2"],
    categories=["High engagement", "Medium engagement", "Low engagement"],
    ordered=True
)

```

```

[235]: cols_summary = [
    'view_count',
    'like_count',
    'comment_count',
    'engagement_score'
]

cluster_summary = (
    data.groupby('engagement_cluster')[cols_summary]
    .mean()
    .round(3)
)

cluster_summary2 = (
    data.groupby('engagement_cluster_phase2')[cols_summary]
    .mean()
    .round(3)
)

display(cluster_summary)
display(cluster_summary2)

```

C:\Users\s1142688\AppData\Local\Temp\ipykernel_13824\2354398264.py:9:
FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

C:\Users\s1142688\AppData\Local\Temp\ipykernel_13824\2354398264.py:15:
FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

	view_count	like_count	comment_count	engagement_score
engagement_cluster				
High engagement	49037.383	841.005	20.173	245.358
Medium engagement	872.947	18.231	1.218	66.648
Low engagement	246.214	2.931	0.101	20.309

	view_count	like_count	comment_count	\
engagement_cluster_phase2				
High engagement	46533.300	799.169	19.063	
Medium engagement	782.404	15.975	1.155	
Low engagement	248.126	2.202	0.118	

	engagement_score
engagement_cluster_phase2	
High engagement	238.922
Medium engagement	61.999
Low engagement	15.269

```
[223]: import plotly.express as px

plot_data = data.copy()

# Alleen voor visualisatie: quantile filtering
low, high = 0.01, 0.9
e_min, e_max = plot_data["engagement_score"].quantile([low, high])
v_min, v_max = plot_data["view_count"].quantile([low, high])

plot_data_filt = plot_data[
    plot_data["engagement_score"].between(e_min, e_max) &
    plot_data["view_count"].between(v_min, v_max)
]

fig = px.scatter(
    plot_data_filt,
    x="view_count",
```

```

y="engagement_score",
color="engagement_cluster",
hover_data=["video_id", "like_count", "comment_count"],
title="Engagement score vs Views per Engagement Cluster",
category_orders={
    "engagement_cluster": ["High engagement", "Medium engagement", "Low_
↪engagement"]
}
)
fig.show()

```

```

[229]: import plotly.express as px

plot_data = data.copy()

# Alleen voor visualisatie: quantile filtering (phase 2)
low, high = 0.01, 0.90
e_min, e_max = plot_data["engagement_score"].quantile([low, high])
v_min, v_max = plot_data["view_count"].quantile([low, high])

plot_data_filt = plot_data[
    plot_data["engagement_score"].between(e_min, e_max) &
    plot_data["view_count"].between(v_min, v_max)
]

fig = px.scatter(
    plot_data_filt,
    x="view_count",
    y="engagement_score",
    color="engagement_cluster_phase2",
    hover_data=["video_id", "like_count", "comment_count"],
    title="Trainingsfase 2 - Engagement score vs Views per Cluster",
    category_orders={
        "engagement_cluster_phase2": [
            "High engagement",
            "Medium engagement",
            "Low engagement"
        ]
    }
)

fig.show()

```

Deze figuur toont de verdeling van de engagement-clusters in trainingsfase 2. De clusters zijn evenwichtig verdeeld en laten geen extreme scheefheid zien. Dit bevestigt dat het toepassen van feature-weging geen onstabiele clustering veroorzaakt.


```
[225]: import plotly.express as px

counts = (
    data["engagement_cluster"]
    .value_counts()
    .rename_axis("engagement_cluster")
    .reset_index(name="count")
)

fig1 = px.bar(
    counts,
    x="engagement_cluster",
    y="count",
    text="count",
    title="Verdeling van Engagement Clusters",
    labels={"engagement_cluster": "Cluster", "count": "Aantal video's"},
    category_orders={"engagement_cluster": ['High engagement', 'Medium engagement', 'Low engagement']}
)

fig1.update_traces(textposition="outside")
fig1.update_layout(template="simple_white", width=900, height=450)
fig1.show()
```

```
[231]: import plotly.express as px

counts_phase2 = (
    data["engagement_cluster_phase2"]
    .value_counts()
    .reset_index(name="count")
    .rename(columns={"index": "engagement_cluster_phase2"})
)

fig2 = px.bar(
    counts_phase2,
    x="engagement_cluster_phase2",
    y="count",
    text="count",
    title="Trainingsfase 2 - Verdeling van Engagement Clusters",
    labels={
        "engagement_cluster_phase2": "Cluster",
        "count": "Aantal video's"
    },
    category_orders={
        "engagement_cluster_phase2": [
            "High engagement",

```

```

        "Medium engagement",
        "Low engagement"
    ]
}
)

fig2.show()

```

```

[ ]: from sklearn.metrics import silhouette_score

sil_phase1 = silhouette_score(X_scaled, labels_phase1)
sil_phase2 = silhouette_score(X_weighted, labels_phase2)

print(f"Silhouette fase 1 (alle features): {sil_phase1:.3f}")
print(f"Silhouette fase 2 (weighted features): {sil_phase2:.3f}")

```

```

[233]: import joblib
import os

os.makedirs("model", exist_ok=True)

# Alles wat je nodig hebt voor inference
model_bundle = {
    "kmeans": kmeans_phase2,
    "scaler": scaler,
    "weights": weights_phase2,
    "features": features
}

joblib.dump(model_bundle, "models/engagement_model.pkl")

print("engagement_model.pkl succesvol opgeslagen")

engagement_model.pkl succesvol opgeslagen

```